

File Organization



File Organization

- **Database** as a collection of **files**.
- **File** a sequence of **records**.
- **Record** a sequence of **fields**.

- One approach:
 - Assume record size is fixed
 - Each file has records of one particular type only
 - Different files are used for different relations

Fixed-Length Records

- Simple approach:
 - Store record i starting from byte $n \times (i - 1)$.
 - Simple record access but records may cross blocks
 - Modification: do not allow records to cross block boundaries

- Deletion of record i :
 1. move records “ $i + 1, \dots, n$ ” to “ $i, \dots, n - 1$ ”
 2. move record n to i
 3. do not move records, but link all free records on a *free list*

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Deleting record 3 and compacting



record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Deleting record 3 and moving last record

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
 record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Free Lists

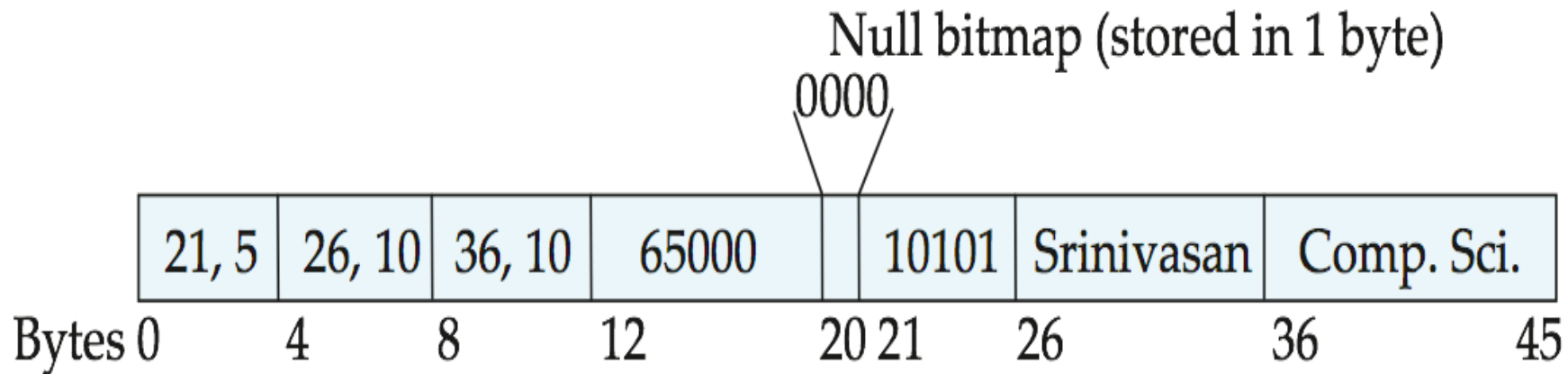
- Store the address of the first deleted record in the file header.
- This first record stores the address of the second deleted record, and so on...
- More space efficient representation

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

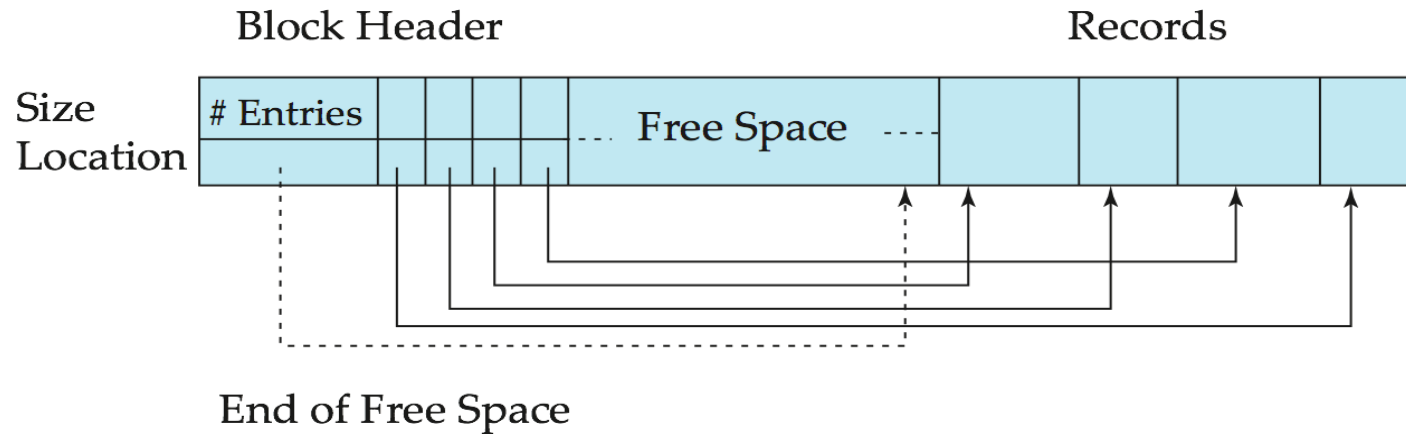
```
graph LR; R0[record 0] --> R1[record 1]; R2[record 2] --> R3[record 3]; R4[record 4] --> R5[record 5]; R6[record 6] --> GND[Ground];
```

Variable-Length Records

- Approaches:
 1. Storage of multiple record types in a file.
 2. Record types that allow variable lengths for one or more fields such as strings, **varchar**
 3. Record types that allow repeating fields (used in some older data models).
- Attributes are stored in order
- Null values represented by null-value bitmap



Variable-Length Records: Slotted Page Structure



- **Slotted page** header contains:
 - Number of record entries
 - End of free space in the block
 - Location and size of each record

Organization of Records in Files

1. Heap

- A record can be placed anywhere in the file where there is space.

2. Sequential

- Store records in sequential order, based on the value of the search key of each record.

3. Hashing

- A hash function computed on some attribute of each record;
- The result specifies in which block of the file the record should be placed.

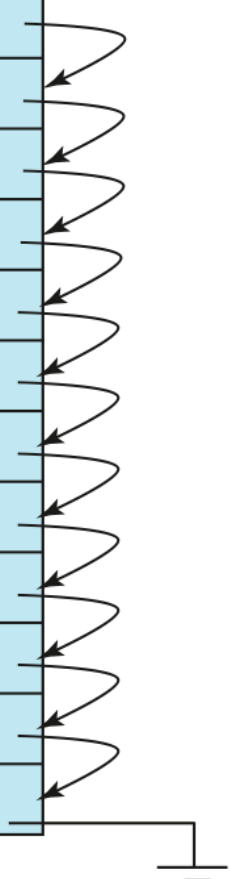
4. Multitable clustering file organization

- Store related records on the same block to minimize I/O

Sequential File Organization

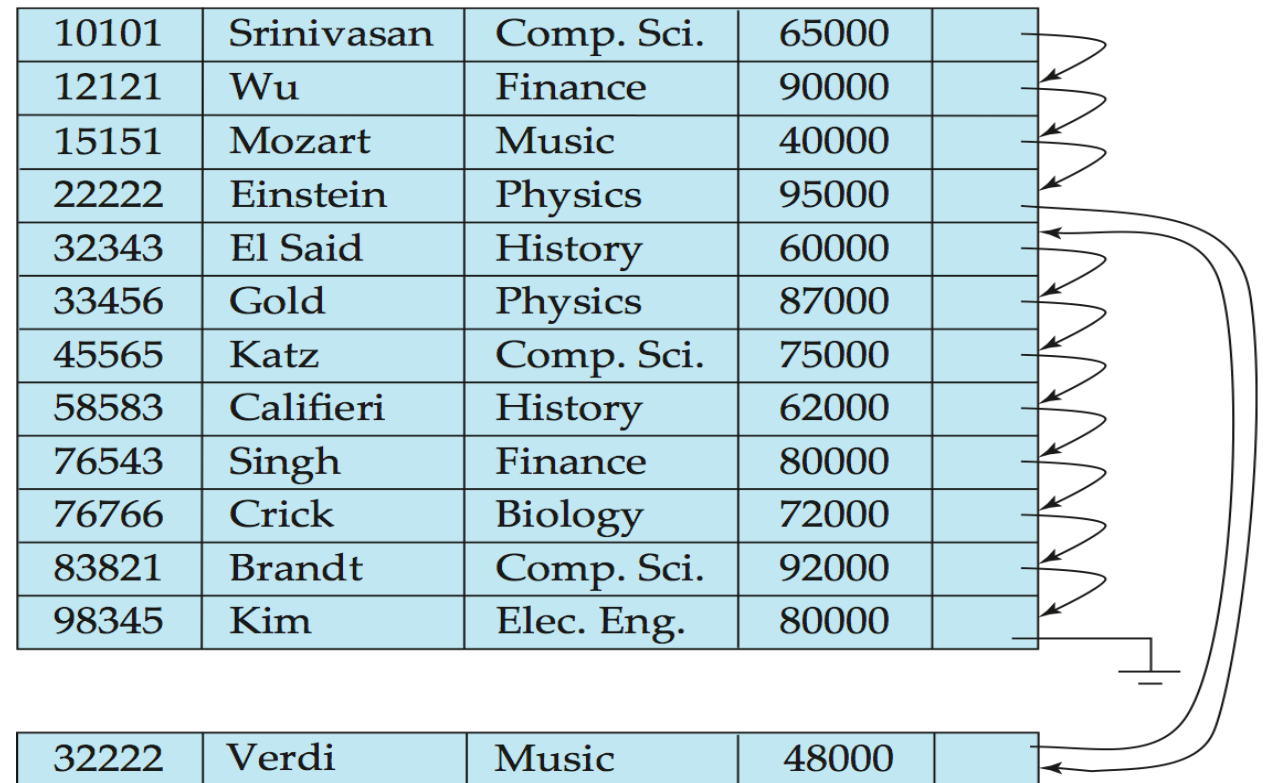
- The records in the file are ordered by a **search-key**

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



Sequential File Organization

- Deletion: use pointer chains
- Insertion: locate the position where the record is to be inserted
- **Overflow block;** if no free space



Multitable Clustering File Organization

- Store several relations in one file using a **multitable clustering** file organization

department

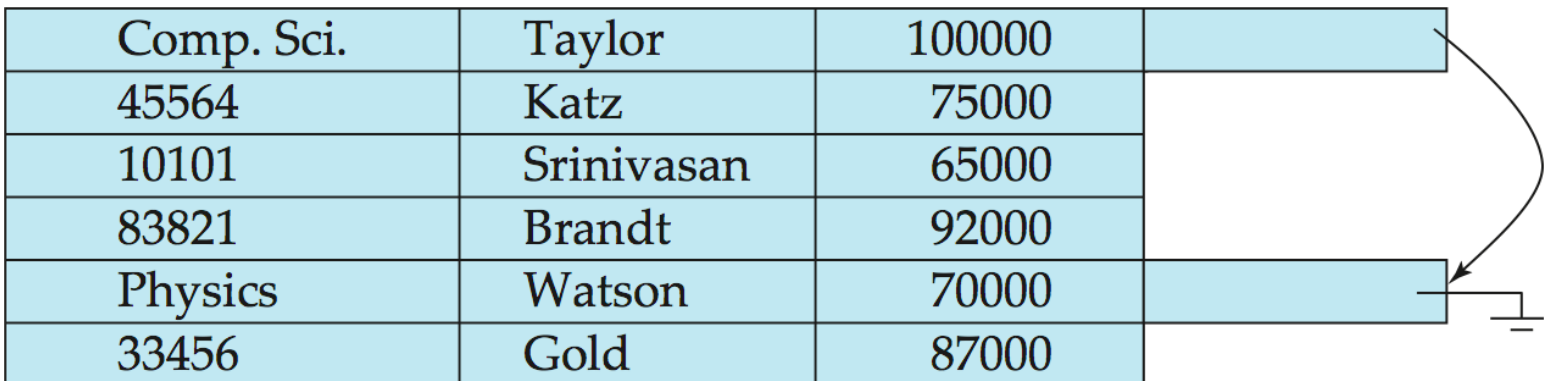
<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Physics	Watson	70000

instructor

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

multitable clustering
of *department* and
instructor

Comp. Sci.	Taylor	100000	
45564	Katz	75000	
10101	Srinivasan	65000	
83821	Brandt	92000	
Physics	Watson	70000	
33456	Gold	87000	

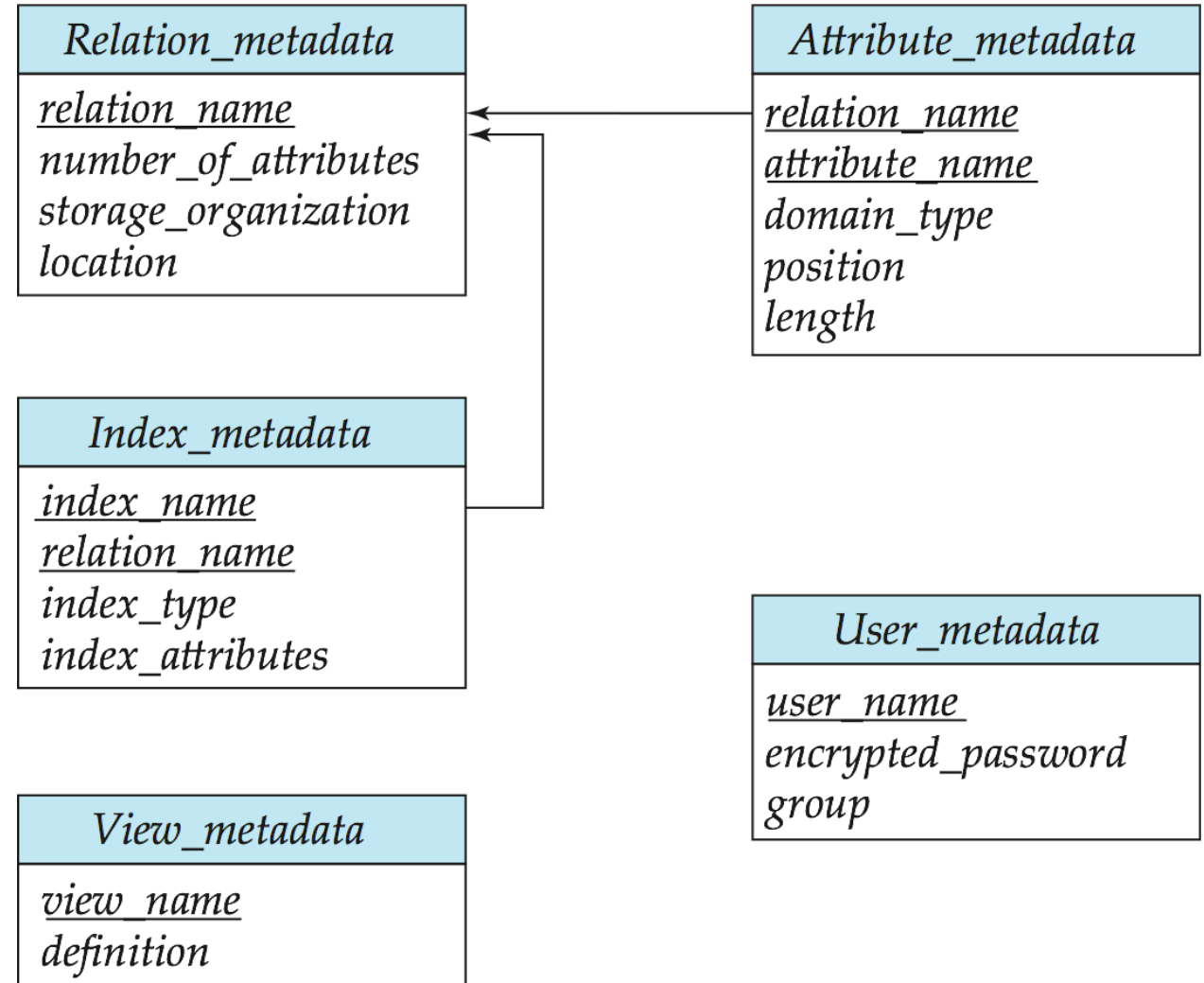


Data Dictionary Storage

- The **Data dictionary** stores **metadata**:
 - Information about relations
 - Names of relations
 - Names, types and lengths of attributes of each relation
 - Names and definitions of views
 - Integrity constraints
 - User and accounting information, including passwords
 - Statistical and descriptive data
 - number of tuples in each relation
 - Physical file organization information
 - How relation is stored (sequential/hash/...)
 - Physical location of relation

System Metadata

- Relational representation on disk
- Specialized data structures designed for efficient access, in memory



Storage Access

- **Buffer**
 - Portion of main memory available to store copies of disk blocks.
- **Buffer manager**
 - Subsystem responsible for allocating buffer space in main memory.



Piran.mj2@gmail.com